

1a Prova de ELT024 – Programação para Sistemas Embarcados 2014

Prof. Rodrigo Maximiano Antunes de Almeida

(20 pts) Questão 1: Configure o registro T0CON de modo que o timer 0 do PIC esteja desligado, operado em 8 bits, ligo ao clock interno com um prescaler de 256 para 1, de modo que sua velocidade seja 256 vezes mais lenta que o clock do processador.

REGISTER 11-1: T0CON: TIMER0 CONTROL REGISTER

R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
TMR0ON	T08BIT	T0CS	T0SE	PSA	T0PS2	T0PS1	T0PS0
bit 7							bit 0

bit 7	TMR0ON: Timer0 On/Off Control bit 1 = Enables Timer0 0 = Stops Timer0
bit 6	T08BIT: Timer0 8-Bit/16-Bit Control bit 1 = Timer0 is configured as an 8-bit timer/counter 0 = Timer0 is configured as a 16-bit timer/counter
bit 5	T0CS: Timer0 Clock Source Select bit 1 = Transition on T0CKI pin 0 = Internal instruction cycle clock (CLKO)
bit 4	T0SE: Timer0 Source Edge Select bit 1 = Increment on high-to-low transition on T0CKI pin 0 = Increment on low-to-high transition on T0CKI pin
bit 3	PSA: Timer0 Prescaler Assignment bit 1 = Timer0 prescaler is NOT assigned. Timer0 clock input bypasses prescaler. 0 = Timer0 prescaler is assigned. Timer0 clock input comes from prescaler output.
bit 2-0	T0PS2:T0PS0: Timer0 Prescaler Select bits 111 = 1:256 Prescale value 110 = 1:128 Prescale value 101 = 1:64 Prescale value 100 = 1:32 Prescale value 011 = 1:16 Prescale value 010 = 1:8 Prescale value 001 = 1:4 Prescale value 000 = 1:2 Prescale value

```
T0CON = 0b010xx111;
```

```
T0CON = 0x47;
```

```
T0CON = 71;
```

(25 pts) Questão 2: Crie um **programa** cíclico que realize a leitura de 3 chaves na porta B, nos bits 1, 2 e 3. Em seguida este programa deverá acender uma quantidade de leds correspondente ao somatório dos números das chaves. Ex: Se as chaves 1 e 3 estiverem pressionadas, 4 leds serão acesos. Se as chaves 1, 2 e 3 estiverem pressionadas 6 leds serão acesos. Se nenhuma das chaves estiver pressionada todos os leds devem ser apagados. As chaves podem ser lidas através da variável PORTB e os leds se encontram na variável PORTD.

```
#include "basico.h"
```

```
#include "config.h"
```

```
void main void
```

```
int chaves,i;
```

```
TRISD = 0x00;
```

```
TRISE = 0x0E;
```

```
for(;;){
```

```
    chaves = 0;
```

```
    if (bitTst(PORTB,1)){
```

```
        chaves+=1;
```

```
    }
```

```
    if (bitTst(PORTB,2)){
```

```
        chaves+=2;
```

```
    }
```

```
    if (bitTst(PORTB,3)){
```

```
        chaves+=3;
```

```
    }
```

```
    PORTD = 0x00;
```

```
    for(i=0;i<chaves;i++){
```

```
        bitSet(PORTD,i);
```

```
    }
```

```
}
```

```
}
```

(25 pts) Questão 3: Crie uma **biblioteca** "alarme" que será responsável por armazenar internamente os limites superiores e inferiores de alarme de temperatura. Esta biblioteca deve possuir as seguintes funções:

- void ConfiguraAlarmeAlto(int valor) que atualizará o valor do limite superior do alarme. A função deve garantir

que o valor não seja mais baixo que o limite inferior.

- void ConfiguraAlarmeBaixo(int valor) que atualizará o valor do limite inferior do alarme. A função deve garantir que o valor não seja mais alto que o limite superior.
- int ChecaAlarme(int temperatura) que receberá o valor de temperatura e, comparando com os limites, retornará se o valor está dentro da faixa (retornar 0), acima do limite superior (retornar 1), abaixo do limite inferior (retornar -1).

```
//alarme.h
#ifndef ALARME_H
#define ALARME_H
void ConfiguraAlarmeAlto(int valor);
void ConfiguraAlarmeBaixo(int valor);
int ChecaAlarme(int temperatura);
#endif

//alarme.c
static int alarmeAlto;
static int alarmeBaixo;
void ConfiguraAlarmeAlto(int valor){
    if(valor > alarmeBaixo){
        alarmeAlto = valor;
    }
}
void ConfiguraAlarmeBaixo(int valor){
    if(valor < alarmeAlto){
        alarmeBaixo = valor;
    }
}
int ChecaAlarme(int temperatura){
    if(temperatura > alarmeAlto){
        return 1;
    }
    if(temperatura < alarmeBaixo){
        return -1;
    }
    //não caiu em nenhum dos ifs, temperatura normal
    return 0;
}
```

(30 pts) **Questão 4:** Uma prensa industrial possui vários sistemas de segurança para evitar que seja acionada de maneira errada. Um deles é um sensor de fim de curso que fica ativo quando a porta está fechada. Um segundo sensor indutivo fica dentro do equipamento e verifica se a peça foi colocada corretamente. Por fim o acionamento é feito utilizando dois botões. Estes botões devem ser pressionados ao mesmo tempo. Faça um **programa** que verifique se os sensores de segurança (fim de curso no bit 4 da porta E e indutivo no bit 7 da porta E) e acione a prensa (bit 5 da porta E) quando os dois botões forem pressionados. Caso os botões forem pressionados em tempos diferentes, o programa deve aguardar os botões serem soltos antes de verificar novamente. (botões bits 2 e 3)

```
#include "basico.h"
#include "config.h"
void main(void){
    TRISE = 0xDF; //apenas bit 5 é saída, 7 4 3 e 2 são entradas
    for(;;){
        //apenas bt 2 ativado, espera os dois serem soltos
        if(bitTst(PORTE,2) && !(bitTst(PORTE,3)){
            while (bitTst(PORTE,2) || (bitTst(PORTE,3));
        }
        //apenas bt 3 ativado, espera os dois serem soltos
        if(!(bitTst(PORTE,2)) && bitTst(PORTE,3){
            while (bitTst(PORTE,2) || (bitTst(PORTE,3));
        }

        //itens de segurança ativados
        if(bitTst(PORTE,4) && bitTst(PORTE,7)){
            //botões pressionados ao mesmo tempo
        }
    }
}
```

```
if(bitTst(PORTE,2) && bitTst(PORTE,3){  
    //ativa prensa  
    bitSet(PORTE,5);  
    bitClr(PORTE,5);  
}  
}  
}
```